ULB ECOLE POLYTECHNIQUE DE BRUXELLES

INFO-H518: Immersive Multimedia Technologies I

Exercises INFO-H518: Part I

Niccolò Benedetto Mat. 626872
Paolo Bordis Alberto Mat. 628838

Prof. Lafruit Gauthier

Prof. Losfeld Armand

November, 2025

Contents

1 Radial distortion removal **2**

1.0.1 Q_R1 2

1.0.2 Q_R2 3

1.0.3 Q_R3 4

1.0.4 Q_R4 6

1.0.5 Q_R5 6

1.0.6 Q_R6 7

2 Homography for panoramic stitching **8**

2.0.1 Q_H1 8

2.0.2 Q_H2 9

2.0.3 Q_H3 9

2.0.4 Q_H4 14

2.0.5 Q_H5 14

3 Rectification **17**

3.0.1 Q_Rec1 17

3.0.2 Q_Rec2 18

3.0.3 Q_Rec3 19

3.0.4 Q_Rec4 21

4 Stereo Matching **23**

4.0.1 Q_S1 24

4.0.2 Q_S2 24

4.0.3 Q_S3 24

4.0.4 Q_S4 25
4.0.5 Q_S5 26

List of Figures

2.1 Snapshot of `DetectCorner` 10
2.2 Snapshot of calling `Corner_Uniqueness` 10
2.3 Snapshot of `CornerPointMatching_NCC` 11
2.4 Snapshot of `Normalization` 12
2.5 Snapshot of `ComputeH` 13
2.6 Input images I_1, I_2, I_3, I_4 used for the four-image panorama. 15
2.7 Inliers for pairs (I_1, I_2), (I_2, I_3) and (I_3, I_4) obtained by RANSAC. 15
2.8 Outliers for pairs (I_1, I_2), (I_2, I_3) and (I_3, I_4) obtained by RANSAC. 15
2.9 Stitched images $stitched_12$, $stitched_123$ and $stitched_1234$ 16

3.1 Epipolar line script output 20
3.2 Epipolar line and measured point 21

4.1 Outcome of stero matching ipolcore demo 23
4.2 Low values parameters result 25
4.3 Disparity map result obtained with filter parameters: Radius = 14, Alpha = 0.7. 26

1 Radial distortion removal

1.0.1 Q_R1

Question: How does the algorithm know that it has converged to a good (or good enough) solution? What is the error function, exactly?

The algorithm determines it has converged when the error function $E(d)$ stabilizes (the relative change between iterations is below a small threshold) or when the gradient magnitude approaches zero. Empirically, we also know the solution is good if the Root Mean Square Error (RMSE) of the point-to-line distances is "sufficiently" small.

The error function (often based on the "plumb-line" constraint) is defined as the sum of squared distances between the undistorted points and their corresponding best-fit straight lines:

$$E(d) = \sum_{j=1}^{N_l} \sum_{i=1}^{N_j} \text{distance}(C_d(x_{ji}), \text{line}_j)^2$$

where:

- $E(d)$ is the energy function we want to minimize.
- d represents the distortion parameters.
- x_{ji} is the i-th detected point belonging to the j-th line segment.
- $C_d(\cdot)$ is the distortion correction function.
- line_j is the estimated straight line for the j-th segment.

To minimize this energy, we compute the gradient $\nabla E(d)$ and use an iterative optimization method (such as Gradient Descent or Levenberg-Marquardt) to update the parameters d in the direction that reduces the error.

1.0.2 Q_R2

Question: Try polynomial and division models. Comment on the difference in results.

Below is the output for the polynomial model optimization:

```
Selected parameters:
         High Canny's threshold: 0.8
         Initial normalized distortion parameter: 0
         Final normalized distortion parameter: 3
         Maximum distance between points and line: 4
         Maximum differente between edge point and line orientations:
            10
         Model applied: pol
         Center optimization: True
-------------------------
Results:
         Number of detected lines: 38
         Total amount of line points: 4167
         Distortion center: (523.516, 364.748)
         Estimated normalized distortion parameters: p1 = 1.56205 p2 =
            0.154645
         Average squared error distance in pixels between line and
            associated points = 1.80434
```

Listing 1.1: Optimization Output (Polynomial Model)

Comparison and Analysis: Comparing these results with the division model, we observed that the polynomial model detected one additional line and a larger total number of line points.

Regarding the parameters and error:

- The first distortion parameter (p_1) for the polynomial model is approximately double that of the division model.
- The final error is higher for the polynomial model ($E_p(d) > E_d(d)$), indicating worse convergence.

Critical Analysis: The higher error in the polynomial model, despite detecting more points, suggests two conclusions:

1. **Model Suitability:** The Division model appears inherently better suited for this specific lens distortion. It often approximates high radial distortion more efficiently than the Polynomial model, achieving a better fit with lower residual error.

2. **Extra points:** The polynomial model may include points that are less well aligned with the straight-line constraint after correction. While the model "found" more data, these points did not align well with the straight-line constraint, effectively increasing the average squared error.

1.0.3 Q_R3

Question: Explain the difference in the principal point position between [A] and [B].

```
Selected parameters:
         High Canny's threshold: 0.8
         Initial normalized distortion parameter: 0
         Final normalized distortion parameter: 3
         Maximum distance between points and line: 4
         Maximum differente between edge point and line orientations:
            10
         Model applied: pol
         Center optimization: True
-------------------------
Results:
         Number of detected lines: 61
         Total amount of line points: 3751
         Distortion center: (563.677, 378.755)
         Estimated normalized distortion parameters: p1 = 0.526388 p2 =
             0.121849
         Average squared error distance in pixels between line and
            associated points = 3.68993
```

Listing 1.2: not cropped

```
Selected parameters:
         High Canny's threshold: 0.8
         Initial normalized distortion parameter: 0
         Final normalized distortion parameter: 3
         Maximum distance between points and line: 4
         Maximum differente between edge point and line orientations:
            10
         Model applied: pol
         Center optimization: True
-------------------------
Results:
         Number of detected lines: 85
         Total amount of line points: 4407
         Distortion center: (479.789, 357.177)
         Estimated normalized distortion parameters: p1 = 0.176136 p2 =
             -0.0151584
         Average squared error distance in pixels between line and
            associated points = 4.53831
```

Listing 1.3: cropped

The difference in the distortion center (principal point) is caused by the cropping of the image.

In the uncropped image [A], the principal point is located near the geometric center of the image, which is the standard assumption for most camera lenses.

In the cropped image [B], the coordinate system has changed. Cropping removes pixels from the borders, which effectively shifts the origin $(0,0)$ of the image. While the physical optical center of the lens remains in the same position relative to the scene content, its pixel coordinates change relative to the new image boundaries.

For example, if we have an image of width $W = 1000$ and the principal point is at $x_c = 500$, cropping 100 pixels from the left side shifts the coordinate system. The new width is $W' = 900$, and the new principal point coordinate becomes $x'_c = 500 - 100 = 400$. This explains why the distortion center in [B] (479.789) is shifted compared to [A] (563.677).

1.0.4 Q_R4

Question: Explain why we see blurry regions in the output result.

The blurry regions in the output are caused by the **resampling** process required to correct the geometric distortion.

When the algorithm un-distorts the image, certain regions must be stretched (upscaled) to move pixels to their corrected positions. In these stretched regions, the bilinear interpolation method estimates new pixel values by averaging the neighboring pixels. This averaging process smoothes out high-frequency details, resulting in a blurry appearance compared to the original sharp regions.

1.0.5 Q_R5

Question: Is the result reliable? Please comment.

```
Selected parameters:
         High Canny's threshold: 0.8
         Initial normalized distortion parameter: 0
         Final normalized distortion parameter: 3
         Maximum distance between points and line: 4
         Maximum differente between edge point and line orientations:
            10
         Model applied: pol
         Center optimization: True
-------------------------
Results:
         Number of detected lines: 46
         Total amount of line points: 5070
         Distortion center: (620, 412.5)
         Estimated normalized distortion parameters: p1 = 0.025 p2 =
            0.00625
         Average squared error distance in pixels between line and
            associated points = 0
```

Listing 1.4:]picture[C]

Yes, the result is reliable and suggests that the input image is likely synthetic or perfectly rectilinear.

The average squared error is exactly 0, which is practically impossible for real-world camera images due to sensor noise and lens imperfections. Combined with the distortion parameters being near zero ($p_1 \approx 0.025$, $p_2 \approx 0.006$), the algorithm has correctly identified that the image is already distortion-free.

1.0.6 Q_R6

Question: Show which kind of radial distortion you get with positive and negative k-values.

The sign of the radial distortion parameter k determines whether the image magnification increases or decreases with the distance from the optical center.

- **Negative k-values ($k < 0$):** Result in **Barrel Distortion**. In this case, image magnification decreases with distance from the optical center. Straight lines appear curved outwards, resembling the shape of a barrel.
- **Positive k-values ($k > 0$):** Result in **Pincushion Distortion**. In this case, image magnification increases with distance from the optical center. Straight lines appear curved inwards.

2 Homography for panoramic stitching

2.0.1 Q_H1

Question: Write down the homography matrix and tell whether these values look reasonable (especially the large and small values might raise questions).

After ruuning the IPOL demo (with default parameters) the algorithm gives us an estimated homography in form of matrix H like that:

$$H = \begin{pmatrix} 1.16688 & -0.113997 & -527.54 \\ 0.177498 & 1.12077 & -63.193 \\ 2.23307 \times 10^{-4} & -2.90808 \times 10^{-5} & 1 \end{pmatrix}. \tag{2.1}$$

Now we know that the homography for an exact affine transformation is give by the equation:

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = \begin{pmatrix} r_1 & r_2 & t_x \\ r_3 & r_4 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \tag{2.2}$$

We immediately note that $H_{33} = 1$. This is telling us the matrix is already normalized. In addition, even if H_{31} and H_{32} are not exact equal to zero, they are very small. So that means this homography is not purely affine, but it has small perspective distortion, which is normal when stiching panoramas. Instead, by concentrating on the top-left 2×2 matrix inside H, we observe that the diagonal is roughly 1 and the off-diagonal is pretty small. And this indicates a plausible rotation/scale. Now, looking at the third column of H that symbolize the image shift (in pixels), we see an horizontal shift of about -528 pixels and a vertical shift of about -63 pixels. If the image

width is on the order of 10^3 pixels that roughly corresponds to moving the second image by half its width to align overlapping region in the stitching panorama.

2.0.2 Q_H2

Question: What is the difference between the "input img 1&2" and "Reg. Image1&2" in the Output Images menu?

In the IPOL demo interface, the panel *Result* displays both "input img 1&2" and "Reg. image 1&2". The two pairs illustrate the effect of the estimated homography.

In particular "input img 1&2" are the exact image used to compose the panorama, without any kind of transformation. Indeed these are in different coordinate system and we can't directly form a panorama. Instead, after estimating the homography H, "Reg. image 1&2" are obtained. Now these two registered images belong to the same coordinate system and are geometrically aligned. In other words we're just saying that the perspective distortion has been compensated by the homography, so now the previous image appear as if taken from the same viewpoint.

2.0.3 Q_H3

Question: Give snapshots of the code where the main processing steps explained in the theory lessons occur, especially for the homography estimation and RANSAC, cf. https://imagine.enpc.fr/moulonp/AC_Ransac.html (it may help to have a look to the code of attached paper _2_Homography_openCV_code_Annex.pdf)

In this question we relate the main theoretical blocks of the homography pipeline (corner detection, feature matching, normalized DLT, and RANSAC) to the implementation provided in the annexed C/OpenCV code. The Homography + RANSAC pipeline from the theory lessons appears in these functions:

(a) `DetectCorner` → **interest point detection.** Uses image gradients to build the 2×2 structure tensor G and computes something called eigenvalues of this to decide if a pixel is a corner. Then calls `Corner_Uniquess` to keep the strongest corner in a neighbourhood.

```
//*************************************************************
// Corner detection
// input:  img
// output: corner (detected corner pts)
// return the total number of detected corner pts
//*************************************************************
int DetectCorner(IplImage *img, CvPoint *corner){
      int num = 0;
      int i,j,ii,jj;
      int height = img->height;
      int width  = img->width;
      int wsize;
      double g11,g12,g22;
      double corner_cost[MAX_CORNERPOINT_NUM];
      double curr_cost;
      CvPoint curr_point;
      CvMat *G = cvCreateMat(2,2,CV_32FC1);
      CvMat *U = cvCreateMat(2,2,CV_32FC1);
      CvMat *V = cvCreateMat(2,2,CV_32FC1);
      CvMat *D = cvCreateMat(2,2,CV_32FC1);
```

Figure 2.1: Snapshot of `DetectCorner`

```
            curr_cost = cvmGet(D,1,1);
            if(curr_cost > T_SMALLEST_EIG)
                  num = Corner_Uniqueness(corner, num, corner_cost, curr_point,
curr_cost);

            if(num >= MAX_CORNERPOINT_NUM){
                  printf("error. MAX_CORNERPOINT_NUM reached!");
                  return -1;
            }
      }
```

Figure 2.2: Snapshot of calling `Corner_Uniqueness`

(b) `CornerPointMatching_NCC` → **putative correspondences** Now it slides a square patch around each candidate extracted in image 2, for each detected corner in image 1. Hence it computes Normalized Cross Correlation (NCC) between them and keeps the match with maximum NCC.

```
                for(ii=-W_SIZE_MATCH; ii<W_SIZE_MATCH; ii++){
                        for(jj=-W_SIZE_MATCH; jj<W_SIZE_MATCH; jj++){
                                if(cur_y+ii < 0 || cur_y+ii >= height || cur_x+jj < 0
|| cur_x+jj >=width)
                                        continue;
                                intensity = cvGet2D(img1, cur_y+ii, cur_x+jj);
                                tmp1 = intensity.val[0] - mean1;
                                intensity = cvGet2D(img2, match_y+ii, match_x+jj);
                                tmp2 = intensity.val[0] - mean2;
                                v1 += tmp1*tmp2;
                                v2 += pow(tmp1, 2.0);
                                v3 += pow(tmp2, 2.0);

                        }
                }
                cur_value = v1 / sqrt(v2*v3);
                if(cur_value > MAX_value)
                {
                        // a better match
                        MAX_value = cur_value;
                        nccvalues[idx] = cur_value;
                        m2[idx].x = (double)match_x;
                        m2[idx].y = (double)match_y;
                        matchedidx[idx] = j;
                }

        }
```

Figure 2.3: Snapshot of `CornerPointMatching_NCC`

(c) `Normalization` → **similarity transform for DLT.** This step is about computing the mean position (centroid) of all points. Hence, through normalization, it computes the average distance to the centroid and derives a scale so that average distance becomes $\sqrt{2}$. Finally it builds a 3×3 matrix T with such scale and translation, and applies $x' = Tx$.

```
                for(ii=-W_SIZE_MATCH; ii<W_SIZE_MATCH; ii++){
                        for(jj=-W_SIZE_MATCH; jj<W_SIZE_MATCH; jj++){
                                if(cur_y+ii < 0 || cur_y+ii >= height || cur_x+jj < 0
|| cur_x+jj >=width)
                                        continue;
                                intensity = cvGet2D(img1, cur_y+ii, cur_x+jj);
                                tmp1 = intensity.val[0] - mean1;
                                intensity = cvGet2D(img2, match_y+ii, match_x+jj);
                                tmp2 = intensity.val[0] - mean2;
                                v1 += tmp1*tmp2;
                                v2 += pow(tmp1, 2.0);
                                v3 += pow(tmp2, 2.0);

                        }
                }
                cur_value = v1 / sqrt(v2*v3);
                if(cur_value > MAX_value)
                {
                        // a better match
                        MAX_value = cur_value;
                        nccvalues[idx] = cur_value;
                        m2[idx].x = (double)match_x;
                        m2[idx].y = (double)match_y;
                        matchedidx[idx] = j;
                }

        }
```

Figure 2.4: Snapshot of `Normalization`

(d) `ComputeH` → **DLT Homography estimation.** Here it builds the A matrix of size $2n \times 9$ from point pairs, according to the homography linear equations. Then runs SVD on A and takes the last row of V (i.e. the smallest singular value) as the vector h. At the end reshapes h into a 3×3 matrix called H. So this step is related directly to the Direct Linear Transform (DLT).

```
void ComputeH(int n, CvPoint2D64f *p1, CvPoint2D64f *p2, CvMat *H)
      int i;
      CvMat *A = cvCreateMat(2*n, 9, CV_64FC1);
      CvMat *U = cvCreateMat(2*n, 2*n, CV_64FC1);
      CvMat *D = cvCreateMat(2*n, 9, CV_64FC1);
      CvMat *V = cvCreateMat(9, 9, CV_64FC1);

    cvZero(A);
      for(i=0; i<n; i++){
            // 2*i row
            cvmSet(A,2*i,3,-p1[i].x);
            cvmSet(A,2*i,4,-p1[i].y);
            cvmSet(A,2*i,5,-1);
            cvmSet(A,2*i,6,p2[i].y*p1[i].x);
            cvmSet(A,2*i,7,p2[i].y*p1[i].y);
            cvmSet(A,2*i,8,p2[i].y);
        // 2*i+1 row
            cvmSet(A,2*i+1,0,p1[i].x);
            cvmSet(A,2*i+1,1,p1[i].y);
            cvmSet(A,2*i+1,2,1);
            cvmSet(A,2*i+1,6,-p2[i].x*p1[i].x);
            cvmSet(A,2*i+1,7,-p2[i].x*p1[i].y);
            cvmSet(A,2*i+1,8,-p2[i].x);
      }

      // SVD
    // The flags cause U and V to be returned transposed
    // Therefore, in OpenCV, A = U^T D V
      cvSVD(A, D, U, V, CV_SVD_U_T|CV_SVD_V_T);

      // take the last column of V^T, i.e., last row of V
      for(i=0; i<9; i++)
            cvmSet(H, i/3, i%3, cvmGet(V, 8, i));

      cvReleaseMat(&A);
      cvReleaseMat(&U);
      cvReleaseMat(&D);
      cvReleaseMat(&V);
}
```

Figure 2.5: Snapshot of `ComputeH`

(e) `RANSAC_homography` → **estimation (RANSAC)** Here we have to run the RANSAC loop for homography estimation: random minimal samples (here selects 4 matches and checks if they are not collinear) → DLT (here normalizes these 4 points through `Normalization`, then estimates homography with `ComputeH`) → inlier counting using `ComputeNumberOfInliers` → keeps the model with largest inlier count → updates the number of iterations N using the standard RANSAC formula

2.0.4 Q_H4

Question: How does the algorithm know that it has converged to a good (or good enough) solution? What is the error function, exactly?

Looking at the code in **_2_Homography_openCV_code_Annex.pdf**, the quality of a homography is evaluated through the *symmetric transfer error*. Given a pair (x, x'), the point x is projected into the second image using H, and x' is projected back using H^{-1}. The forward and backward distances are

$$d_f = \|Hx - x'\|, \qquad d_b = \|H^{-1}x' - x\|.$$

The error used by RANSAC is then

$$E(x, x') = d_f^2 + d_b^2,$$

which is exactly the criterion implemented in `ComputeNumberOfInliers`. A correspondence between x and x' is an inlier when $E(x, x') < T_{\text{DIST}}$ (i.e. the treshold used).

During the RANSAC loop, each candidate homography provides a number of inliers. From the best current inlier ratio w, the algorithm updates the required number of iterations according to the classical formula

$$N = \frac{\log(1-p)}{\log(1-w^s)},$$

with $s = 4$ for homography estimation and p the desired confidence (0.99). As soon as the current iteration index exceeds this value of N, the algorithm think as the probability that a better model still exists becomes negligible. So it stops and returns the homography with the largest inlier count. in other words we can say that the algorithm believes it has found a good enough solution when more iterations would almost certainly not improve the result.

2.0.5 Q_H5

Question: Capture 4 images for yourself and stitch them together into a Panoramic image using this program multiple times for each successive pair of images. Show the intermediate results of inliers and outliers, the homography matrices and the stitched image.



Figure 2.6: Input images I_1, I_2, I_3, I_4 used for the four-image panorama.

We used the IPOL homography demo to estimate homographies between the three pairs (I_1, I_2), (I_2, I_3) and (I_3, I_4). For each pair, SIFT features are detected and matched, RANSAC selects a set of inliers, and the final homography is refined on these inliers. The following figures show the inliers (green) and outlier (red) for the three pairs.

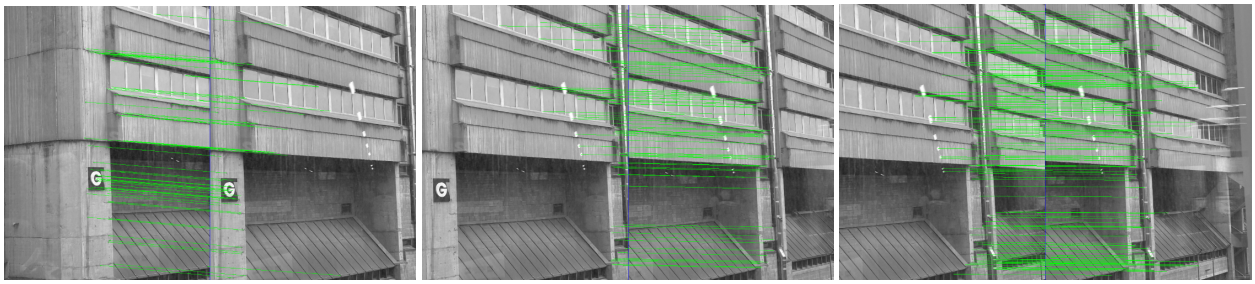


Figure 2.7: Inliers for pairs (I_1, I_2), (I_2, I_3) and (I_3, I_4) obtained by RANSAC.



Figure 2.8: Outliers for pairs (I_1, I_2), (I_2, I_3) and (I_3, I_4) obtained by RANSAC.

The corresponding homography matrices (mapping the left image of each pair onto the right one) are listed in Table 2.1.

Pair	Homography $H_{i\rightarrow i+1}$
(I_1, I_2)	$\begin{pmatrix} 1.03993 & 0.00165037 & -450.052 \\ 0.0265226 & 1.02233 & 43.2944 \\ 3.66939e-05 & -5.04854e-06 & 1 \end{pmatrix}$
(I_2, I_3)	$\begin{pmatrix} 1.035258 & 0.00267532 & -353.421 \\ 0.0199909 & 1.01873 & 7.30925 \\ 2.81357e-05 & -2.11445e-06 & 1 \end{pmatrix}$
(I_3, I_4)	$\begin{pmatrix} 1.02722 & -0.00548468 & -288.277 \\ 0.0230996 & 1.01768 & -10.9004 \\ 2.20153e-05 & 5.08542e-07 & 1 \end{pmatrix}$

Table 2.1: Homography matrices estimated for each neighbouring pair.

At this point in order to obtain the complete stitched image we ran the IPOL algorithm by following these steps:

1. inputs: I_1, I_2 → output: $stitched_12$;

2. inputs: $stitched_12$, I_3 → output: $stitched_123$;

3. inputs: $stitched_123$, I_4 → output: $stitched_1234$;

Where the panorama images yield are listed below:



Figure 2.9: Stitched images $stitched_12$, $stitched_123$ and $stitched_1234$

3 Rectification

3.0.1 Q_Rec1

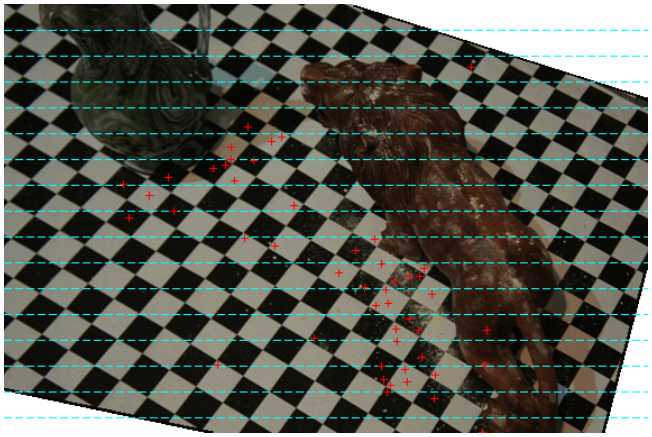
Question: Make an animated gif from your output images to obtain a switching effect between the [D,E] image pair.

Q_Rec1: Animated GIF of the rectified pair

- We applied the IPOL implementation of the quasi-Euclidean epipolar rectification of Fusiello–Irsara to the stereo pair $[E]_lion_right$, $[D]_lion_left$ obtaining two rectified images $E_rect.png$ and $D_rect.png$ of identical size.
- We then created an animated GIF by using ImageMagick via the command:

```
convert -delay 60 -loop 0 E_rect.png D_rect.png rectified_switch.gif.
```

- We observe that in the resulting switching effect image the points of the scene move almost purely horizontally between the two views. We can claim that the absence of vertical change confirms that the rectification has aligned the epipolar lines successfully by removing vertical parallax.



3.0.2 Q_Rec2

Question: Find the homography matrix that transforms the right input image into the rectified right output image. Show that the numerical values indeed yield the input-to-output transformation of the right image.

To verify the right-image homography H, we take the point $(u, v) = (318.4, 168.4)$ (via GIMP analysis) in the original right image and apply the mapping in homogeneous coordinates

$$x = (u, v, 1)^T, \qquad x' = Hx = (u', v', w')^T.$$

We have to use the matrix provided by the IPOL demo, which is something like

$$H = \begin{pmatrix} 1.09428 & -0.254074 & 10.949 \\ 0.279043 & 0.960231 & -72.2937 \\ 0.000287994 & -7.49606 \cdot 10^{-5} & 0.895008 \end{pmatrix},$$

Computing by using this matrix we obtain numerically

$$x' \approx (316.58,\ 178.26,\ 0.9741)^T, \quad (u_{\text{rect}}, v_{\text{rect}}) = \left(\frac{u'}{w'}, \frac{v'}{w'}\right) \approx (325.0,\ 183.0).$$

Using GIMP again we tried our best to measured the corresponding point of (u, v) in the rectified right image. In pixels coordinates we observe that this is given by $(326.5, 183.5)$.

So now, by computing the difference:

$$\Delta u = 326.5 - 325.0 \approx 1.5 pixels$$

$$\Delta v = 183.5 - 183.0 \approx 0.5 pixels$$

and the Euclidean error as well:

$$\sqrt{\Delta u^2 + \Delta v^2} \approx \sqrt{1.5^2 + 0.5^2}$$

we achieve an error of about 1.6 pixels. The discrepancy is quite small, and compatible with human clicking imprecision and interpolation/resampling made during homography application.

3.0.3 Q_Rec3

Question: Give the Fundamental matrix and show for a feature point of your choice in the left image the epipolar line in the right image.

The fundamental matrix between the original left and right image input provided by the IPOL demo appears as follow:

$$F = \begin{pmatrix} 3.2525\times10^{-7} & 2.21378\times10^{-6} & -7.05194\times10^{-4} \\ -2.07754\times10^{-6} & 3.79865\times10^{-7} & 2.51439\times10^{-5} \\ 5.22987\times10^{-4} & -2.74733\times10^{-4} & 4.89222\times10^{-2} \end{pmatrix}.$$

The epipolar constraint follows the IPOL convention

$$X_l^{\mathsf{T}} F X_r = 0,$$

where X_l and X_r denote homogeneous pixel coordinates in the left and right images, respectively.

We choose a feature point in the left image by observing the point pairs from the demo output:

$$X_l = (u_l, v_l, 1)^{\mathsf{T}} = (404.769, 274.674, 1)^{\mathsf{T}}.$$

The corresponding epipolar line in the right image is given by

$$\ell_r = F^{\mathsf{T}} X_l = \begin{pmatrix} a \\ b \\ c \end{pmatrix} = \begin{pmatrix} 8.39919\times10^{-5} \\ 7.25676\times10^{-4} \\ -2.29612\times10^{-1} \end{pmatrix}.$$

Hence, the epipolar line equation in the right image is

$$a\,u + b\,v + c = 0 \quad\Longleftrightarrow\quad 8.40\times10^{-5}\,u + 7.26\times10^{-4}\,v - 2.29612\times10^{-1} = 0.$$

We now took the corresponding point in the right image as pair displayed in the output files from the demo

$$X_r^{\text{meas}} = (395.267, 279.427, 1)^{\mathsf{T}}.$$

Evaluating the epipolar constraint yields

$$|a u_r + b v_r + c| \approx 6.36\times10^{-3}.$$

The geometric distance from the measured point to the epipolar line is

$$d = \frac{|au_r + bv_r + c|}{\sqrt{a^2 + b^2}} \approx 8.71 \text{ pixels.}$$

We can retain this error plausible since we have to consider the noisy nature of fundamental matrix estimation.

In order to draw the epipolar line in the right image we wrote a python script that exploits the fact that the line in the right image can be written as:

$$\ell_r = F^T X_l = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

and any point in the (u, v) in such line satisfies

$$au + bv + c = 0$$

Indeed that's exactly what our code does:

```
X_l = np.array([u_l, v_l, 1.0], dtype=np.float64)

l_r = F.T @ X_l

a, b, c = l_r
```

The script output is something like:

```
Epipolar line coefficients (a, b, c): 8.399189529000012e-05 0.00072567555583 -0.22961209459739998
Line equation in right image: 8.399189529000012e-05 * u + 0.00072567555583 * v + -0.22961209459739998 = 0
Algebraic error at measured point: 0.006360473417101942
Perpendicular distance from point to line: 8.707 pixels
Line endpoints on image border: (0, 316) (573, 250)
ight_epipolar.pngon to: \Download
Displaying the fundamental matrix [[ 3.25250e-07  2.21378e-06 -7.05194e-04]
 [-2.07754e-06  3.79865e-07  2.51439e-05]
 [ 5.22987e-04 -2.74733e-04  4.89222e-02]]
```

Figure 3.1: Epipolar line script output

and the display epipolar line appears as:



Figure 3.2: Epipolar line and measured point

3.0.4 Q_Rec4

Question: Find the extrinsics (R,t) from the left to the right camera image.

From the IPOL demo we obtain the fundamental matrix

$$F = \begin{pmatrix} -3.97\times10^{-7} & -2.68\times10^{-6} & 8.81\times10^{-4} \\ 2.51\times10^{-6} & -4.63\times10^{-7} & 2.34\times10^{-5} \\ -6.62\times10^{-4} & 2.89\times10^{-4} & -5.96\times10^{-2} \end{pmatrix}.$$

But to obtain the roation and translation we need a relationship in term of camera coordinates, not only pixels based given by the fundamental matrix F. To switch from pixels to camera coordinates we use the camera intrinsics K:

$$x_{camera} = K^{-1} x_{pixel}$$

We know that under the Fusiello–Irsara camera model both cameras share the same intrinsics with square pixels and principal point at the image centre. So with focal length $f = 901.458$ and image size 574×382 ($x_c \times y_c$), the calibration matrix is

$$K = \begin{pmatrix} f & 0 & \frac{x_c}{2} \\ 0 & f & \frac{y_c}{2} \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 901.458 & 0 & 287 \\ 0 & 901.458 & 191 \\ 0 & 0 & 1 \end{pmatrix}.$$

And the essential matrix, given by the following equation, is exactly the fundamental matrix exp-

resed in camera coordinates:

$$E = K^{\mathsf{T}} F K.$$

From the theory we know that the real E should have singular values $(s, s, 0)$. But since E comes from noisy correspondences, such computed matrix is not perfect. So we have to fix it by projecting it onto the space of valid essential matrices. In other words we enforce the rank-2 constraint on E by SVD projection to $\mathrm{diag}(\sigma, \sigma, 0)$, we obtain E' and decompose it as $E' = U\,\mathrm{diag}(\sigma, \sigma, 0)\,V^{\mathsf{T}}$. Using

$$W = \begin{pmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix},$$

the two possible rotations are

$$R_1 = UWV^{\mathsf{T}}, \qquad R_2 = UW^{\mathsf{T}}V^{\mathsf{T}},$$

and the translation direction is given by the third column of U.

Numerically (using `Python/Numpy`) since the matrix E is something like

$$E \approx \begin{pmatrix} -0.3226 & -2.1798 & 0.2297 \\ 2.0392 & -0.3763 & 0.5905 \\ -0.2675 & -0.5135 & 0.0038 \end{pmatrix}$$

and

$$E' \approx \begin{pmatrix} -0.3531 & -2.1165 & 0.2125 \\ 2.1032 & -0.3405 & 0.6022 \\ -0.2810 & -0.5009 & -0.0016 \end{pmatrix}$$

then (we report only the computation for R_1)

$$R_1 \approx \begin{pmatrix} 0.9864 & -0.1616 & 0.0292 \\ 0.1619 & 0.9868 & -0.0076 \\ -0.0276 & 0.0122 & 0.9995 \end{pmatrix}, \qquad t \propto \begin{pmatrix} -0.2428 \\ 0.0883 \\ 0.9660 \end{pmatrix}.$$

4 Stereo Matching

To get a good result we need to set this parameters:

- $r^{GF} = 17$
- $\alpha = 0.8$
- check left to right

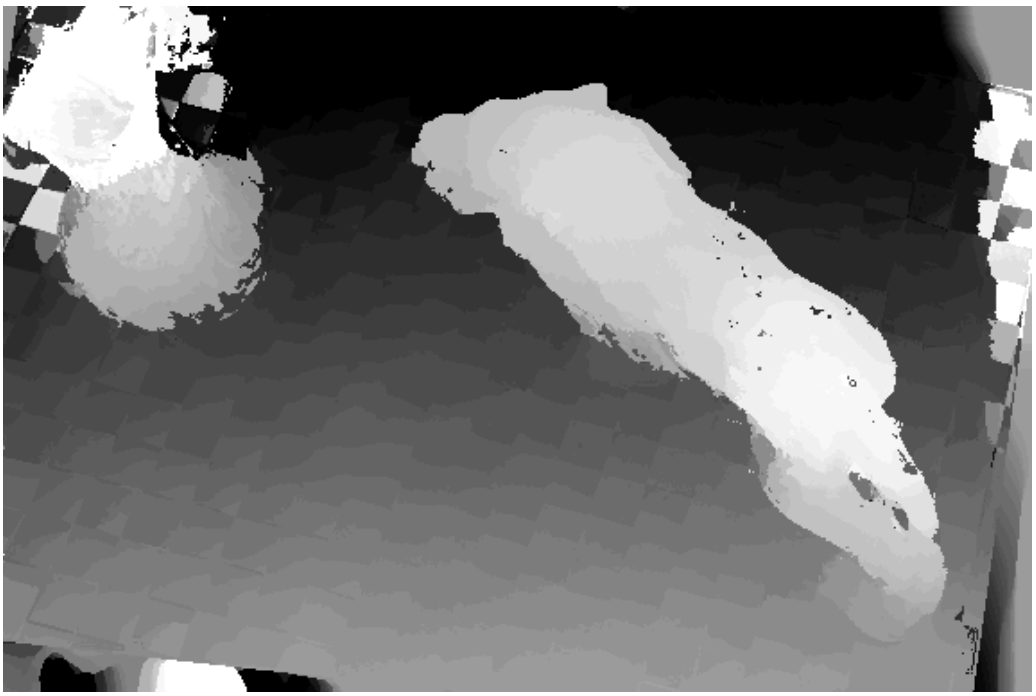


Figure 4.1: Outcome of stero matching ipolcore demo

The output looks like similar of the disparity provided using this tuning parameters.

4.0.1 Q_S1

Question: Why is the result always bad at the boundaries?

The result is bad at boundaries mainly because increasing the radius of the guided filter to clean the disparity map (removing wrong disparities) causes oversmoothing, which generates errors specifically on object boundaries . Another reason is the fattening effect, where neighboring background points are incorrectly labeled as belonging to the foreground object, making the object appear wider than it actually is.

4.0.2 Q_S2

Question: Is this a depth map, or a disparity map?

It is a disparity map. The algorithm computes the displacement (disparity) between the left and right images. While related to depth, the raw output represents pixel shifts, where brighter values typically indicate closer objects (higher disparity) and darker values indicate background (lower disparity).

4.0.3 Q_S3

Question: Why does the vase look so crappy, compared to the lion?

The vase looks crappy because the configuration of $\alpha = 0.9$ heavily favors the gradient term over the color term (0.9 vs 0.1). Since the vase is smooth and has no texture, it contains no useful gradient information. Consequently, the algorithm ignores the reliable color data and instead tries to match based on the gradient cost, which is useless/noisy in that region.

Conversely, if we set the parameters to extremely low values, such as $\alpha = 0.1$ and $r^{GF} = 2$, we obtain the result shown below, where the vase is significantly more defined.

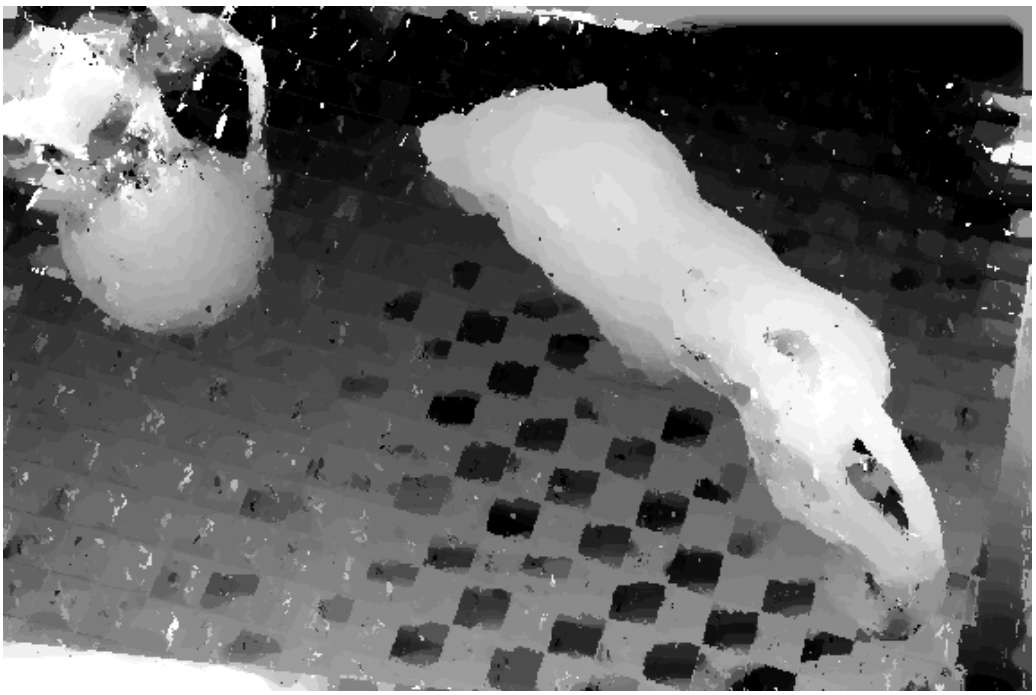


Figure 4.2: Low values parameters result

4.0.4 Q_S4

Question: Redo the experiment with your own pair of input images (be careful in choosing the right parameters and checking the right radio buttons). This result will be used in the following part of the exercises (Part II), where you will have to write your own code for view synthesis



Figure 4.3: Disparity map result obtained with filter parameters: Radius = 14, Alpha = 0.7.

The result shown in Figure 4.3 illustrates the disparity map after applying the post-processing filter. The specific configuration used for this result includes a kernel *radius of 14* and a regularization parameter **alpha (or lambda) of 0.7**.

This parameter set was chosen to achieve a trade-off between smoothness and detail: a larger radius helps to fill the occlusion holes and reduce speckle noise on the skin surface, while the alpha value ensures that the main facial features and depth discontinuities are preserved without excessive blurring.

4.0.5 Q_S5

Question: Explain the main algorithmic steps of the paper, with a focus on the additional terms – compared to the theory lessons - in the Cost function to optimize. Do not spend too much effort in explaining the guided filter and the box filter; rather focus on the cost function, its optimization and the final steps of the algorithm.

Question: Explain the main algorithmic steps of the paper, with a focus on the additional terms – compared to the theory lessons - in the Cost function to optimize. Do not spend too much effort in explaining the guided filter and the box filter; rather focus on the cost function, its optimization and the final steps of the algorithm.

The algorithm proposed by Rhemann et al. (and implemented by Tan and Monasse) estimates the disparity map through a *Cost Volume Filtering* approach. Instead of performing a global optimization (like Graph Cuts) or a purely local block-matching, it computes a per-pixel matching cost, filters this cost volume using a **Guided Filter**, and finally selects the best disparity locally.

The main algorithmic steps are detailed below:

1. The Robust Cost Function Unlike standard approaches that might use a simple Sum of Absolute Differences (SAD) or Squared Differences (SSD) on pixel intensities, this paper introduces a robust cost function composed of two specific terms to handle radiometric distortions and textureless regions:

- **Truncated Color Difference:** To be robust against outliers, the Absolute Difference (AD) of the RGB color channels is averaged and truncated by a threshold τ_1.
- **Truncated Gradient Difference:** To handle illumination changes and lend more weight to textured areas (edges), the absolute difference of the gradients in the x-direction (horizontal derivative) is computed and truncated by a threshold τ_2.

The final cost $C(i,d)$ for a pixel i at disparity d is a weighted linear combination of these two terms, balanced by a parameter α:

$$C(i,d) = (1-\alpha) \cdot \min(||I_L(i) - I_R(i+d)||, \tau_1) + \alpha \cdot \min(|\nabla_x I_L(i) - \nabla_x I_R(i+d)|, \tau_2) \quad (4.1)$$

The inclusion of the gradient term and the truncation of both terms are the key additions compared to basic matching costs, providing greater robustness against noise and lighting variations].

2. Cost Aggregation (Filtering) This raw cost volume is noisy. The core contribution of the paper is smoothing these costs using a **Guided Filter**, which uses the left color image as a guidance map to preserve edges while smoothing the cost slices.

3. Optimization (Winner-Take-All) After filtering, the optimization strategy is purely local. A **Winner-Take-All (WTA)** strategy is applied: for each pixel i, the disparity d associated with the minimum aggregated cost is selected[cite: 474]:

$$d(i) = \arg \min_{d' \in [d_{min}, d_{max}]} \{C^{filt}(i, d')\} \tag{4.2}$$

4. Refinement Steps (Post-Processing) The raw disparity map produced by WTA contains errors and occlusions. The final steps of the algorithm address these issues:

- **Left-Right Consistency Check:** The disparity is computed from Left-to-Right and Right-to-Left. Pixels where the two estimates do not match (based on a threshold d_{LR}) are invalidated and marked as occlusions.
- **Occlusion Filling (Densification):** Invalid pixels are filled using the disparity of the closest non-occluded neighbor from the background (assuming the background is locally constant behind the occluder).
- **Weighted Median Filtering:** Finally, to remove streak artifacts and smooth the map while preserving edges, a Weighted Median Filter (WMF) is applied. This filter uses bilateral weights (spatial and color distance) derived from the reference image.